Data

CNP_ERD.html Download and open in a browser

Domain Review

There are 25 Domains Across 4 Authority Tiers.

1. The Data Spine (purple) sits at the top. ProgramGroupYears is the universal FK chain everything else hangs off.
2. Tier 1 (black) holds the four domains that decide money, eligibility, and enforcement — nothing else in the system makes those calls.
3. Tier 2 (grey) constrains inputs but never decides outcomes.
4. Tier 3 (blue) owns program-specific rules and operational modules.
5. Tier 4 (green) consumes facts but never influences behavior.

387 tables live in one shared cnp_global database; 1,015 tables live in a separate cnp_ops database per state. The database itself is the tenant boundary so no TenantId column is needed.

Data Spine

Program Registry · ProgramGroupYears

1 · Authoritative

Claims & Reimbursement · Accounting & Disbursement · Compliance & Review · Eligibility & Direct Certification

2 · Supporting

Identity & Access · Configuration & Tenant · Sponsor & Site Registry · Applications & Agreements · Program Registry (DATA SPINE)

3 · Specialist — own program rules & integration surfaces

Programs: SNP · CACFP · DCH · SFSP · FFVP · SSFO · CEP + 7 Module Domains + 4 Platform Services

4 · Observational

Reporting & Analytics (→ Snowflake) · Audit & Observability · Notifications · Reference Data & Transparency · LaunchDarkly

The Problems We're Solving

Reading the legacy CTS and DIS repositories revealed five patterns that define why the current system can't scale:

The same logic forks by state.

Missouri's federal report BLL is already 50% larger than Utah's — same form, same federal requirement, different file because state-specific logic migrated into code instead of configuration. Every state added makes the next one harder to maintain.

Business rules exist but don't run

The CACFP Institution Management Plan has over 80 compliance rules fully coded. The entire validation method is commented out. Only one field is checked at submission. States have been operating without that enforcement for years — not because the rules don't matter, but because they were never turned on.

Schema versions live in application code

Hawaii's DC application tracks which database schema version it's talking to using hardcoded year constants that branch SQL queries. Three schema versions co-exist in production today. The code — not migrations, not the database — is managing the difference.

State-specific data is embedded in source code

SFTP IP addresses, plaintext credentials, and PGP key file paths are string literals and app.config values. There was no configuration system to put them in, so they went into code. Every state deployment made that worse.

Business logic is in the database and diverging

SQL CLR functions handle name matching. Custom sequence generators manage key allocation. Stored procedures encode logic that runs outside application visibility. The database has become a second application layer — one that's harder to test, harder to version, and harder to move.

What We Need to Finish It

The design was built from what I could read. I couldn't access some of the most important logic.

Several stored procedures were encrypted (deliberately, and correctly) to protect LINQ's intellectual property after incidents where states attempted to modify or redistribute our software. That was the right call. **The consequence is that the offshore team cannot characterize or reimplement what those procedures do without seeing them first.**

Before those modules can be assigned to a sprint, we need two things:

1. **Decrypted procedure source** so the logic can be characterized and tested before migration (cloud ops?)
2. **Engineer knowledge from this team** the people who built those procedures, who know the edge cases, who remember why a particular branch exists.

The offshore team can build from design documents. They cannot build from institutional memory they don't have access to. The internal team's involvement in characterizing this logic is the only way it can come together correctly.

DATA SPLIT

387 tables live in one shared cnp_global database; 1,015 tables live in a separate cnp_ops database per state. The database itself is the tenant boundary so no TenantId column is needed.

cnp_global 387 tables shared across all states	**cnp_ops** 1,015 tables per state
•ClaimRateAmounts (effective-dated)	•Sponsors · Claims · Reviews
•All *Types / *Codes / *Statuses	•Applications · Users · Eligibility
•ReviewVersions · VCAScoringTools	•All operational per-state data
•AnnualFormTypes	•One schema = one tenant

*All *Types, *Codes, *Statuses + ProgramMealRates + ReviewVersions + VCAScoringTools live in cnp_global. Operational data (Sponsors, Claims, Reviews, Applications, Users) lives in cnp_ops — one schema per state.*

DESIGN DECISIONS

1. **Eligibility snapshot is frozen at approval**: The eligibility snapshot (enrollment counts, rate tier qualifications, site program configuration) is captured from the approved application and never modified. Dollar rates are not stored in ClaimLaws, they are resolved at claim calculation time from ClaimRateAmounts using the claim date as the effective date key. Immutable record of what was decided about eligibility, not about rates.
2. **ProgramGroupYears is the data spine**: Every entity FKs through ProgramGroupYears. It is the universal join key across all domains.
3. **Database IS the tenant boundary**: No TenantId column in cnp_ops. Schema name = state = tenant. Clean isolation by design.
4. **JSONB for rules and configuration**: Rules, overrides, and configuration stored as JSONB — eliminates schema forking for state-specific logic.
5. **Federal SPs to PostgreSQL functions**: Characterization tests required before any translation. Stored procedures encode business logic that may be undocumented.

Example



An Annual Rate Update That Doesn't Require a Deployment

USDA FNA publishes SY2026-27 rates · June 2026 · First proof-of-concept of the new architecture

NSLP, SBP, and CACFP rates follow a July 1 – June 30 cycle. SFSP rates follow a January 1 – December 31 cycle and are published in the Federal Register in January or February each year. The test: does one database INSERT handle all 22 states simultaneously, without touching any application code, without any SP updates, without any per-state deployment?

1. **Federal Register publication detected.** June 12, 2026: USDA FNA publishes "National School Lunch Program — National Average Payments SY2026-27" (Document FNA-2026-NP-0047). The AI

Regulatory Detection service, monitoring the Federal Register API daily, flags the document within 4 hours. Classification: Type 1 Rate Change, programs NSLP/SBP/SFSP/CACFP, effective July 1, 2026. Alert sent to the platform team with source document link and affected program list.

2. **LLM extracts the rate table.** The rate extraction service feeds the Federal Register PDF to the LLM with a structured extraction prompt. Two minutes later, the output is a typed JSON object: `{ "LunFrRte": 4.28, "LunRdcRte": 3.97, "LunPdRte": 0.53, "BrkFrRte": 2.39, "PbrRte": 0.09, "SnlFrRte": 4.46 … }`. The platform engineer compares it line-by-line against the published document.
3. **Human review and anomaly check.** Delta from SY2025-26: free lunch up $0.16 (+3.9%), paid lunch up $0.03 (+6%). No rate change exceeds 10% — no anomaly gate triggered. Review takes 12 minutes. Engineer approves.
4. **Single INSERT. All 22 states. Zero deployments.**

sql

```sql
INSERT INTO ClaimRateAmounts
  (ProgramCode, MealType, EligCategory, Rate, EffectiveDate)
VALUES
  ('NSLP', 'Lunch', 'Free',    4.28, '2026-07-01'),
  ('NSLP', 'Lunch', 'Reduced', 3.97, '2026-07-01'),
  ('NSLP', 'Lunch', 'Paid',    0.53, '2026-07-01'),
  ('NSLP', 'Breakfast', 'Free', 2.39, '2026-07-01')
  -- ... 14 more rows
```

This runs against cnp_global — the one shared database that all 22 state application instances read. No code change. No SP modification. No deployment to any state. Total time from detection to rate loaded: 16 minutes.

5. **New applications freeze the new rate.** December 3, 2026: state agency approves it. `ClaimLaws.FreezeOnApproval()` runs — reads the approved application and captures the eligibility snapshot: enrollment counts, Severe Need qualification, CEP ISP percentage, Performance-Based eligibility, site program configuration. ClaimLaws record #9144 created with the eligibility tier data. No dollar rate is stored in ClaimLaws. When Weber's first claim is processed, the claim engine looks up the rate from `ClaimRateAmounts` using the claim date as the effective date key — returns $4.28 for SY2026-27.
6. **Old ClaimLaws are untouched.** July 2026: Salt Lake City SD submits a revised October 2025 claim (late revision, approved by state agency under Utah's late claim override policy). Validation loads ClaimLaws #8821 — the record created when their SY2025-26 application was approved in October 2024. LunFrRte = **4.12**. The July rate update did not touch ClaimLaws #8821. The system uses $4.12 for the revised October 2025 claim. Correct.
7. **Two states, two rates, both right simultaneously.** Weber's first SY2026-27

claim in October 2026 processes at $4.28. Salt Lake City's revised SY2025-26 claim in July 2026 processes at $4.12. Both are correct. Both trace back to a single ClaimLaws record that was frozen at the right time. No manual intervention. No phone calls to check which rate to use. No SP deployment.

8. **FDCH tier certification windows are evidence-based, not configurable:** Tier I certification based on census tract or school eligibility = 60 months (IsFederal=Yes). Tier I based on provider income = 12 months (IsFederal=Yes). Neither window can be shortened by state config. Tracked in FDCHTierCertifications and FDCHMonthlyTierIndicators (one row per provider app, with Oct–Sep monthly indicators).
9. **Corrective action is an append-only dialog, not a text field:** ReviewCorrectiveActionMasters links to a finding; ReviewCorrectiveActionDetails records each back-and-forth entry between reviewer and sponsor. Neither party can edit prior entries — only append. This is the record of remediation.
10. **ReviewFindings severity-3 is a state agency override:** Severity 1 = blocks submission. Severity 2 = warning only. Severity 3 = state agency override that forces the record back to PendingApproval without blocking the submission path. It is not a federal concept — it exists only in state-configured workflows.

Data Spine

Every piece of data in this platform has to answer two questions:

1. which federal program is this for
2. which school year does it cover?

This section is the single source of truth for both. When USDA publishes a new school year starting October 1, this is where that year is registered. Everything else in the system links back here so those two questions always have a consistent, reliable answer.

The data spine immediately communicates what ProgramGroups → ProgramGroupYears → ProgramGroupEnrollments does: every Application and every Claim FK-chains through ProgramGroupYears. Without this chain we cannot build a correctly joined data model.

ProgramGroups

Master list of federal programs the platform supports

Each row is essentially a program code with a human name. Examples:

NSLP

SBP

SFSP

This table almost never changes — a new row would only appear if the USDA created an entirely new federal program. It lives in **cnp_global** because every state reads from the same program definitions.

ProgramGroupYears

One row exists for each combination of program + year. The program year boundary depends on the program:

- **NSLP, SBP** — StartDate = July 1, EndDate = June 30 (school year, e.g. SY2025-26)
- **CACP** — StartDate = October 1 , EndDate = September 30
- **SFSP** — Seasonal (May - September operational window)

This is *not* the Federal Fiscal Year (October 1 – September 30). The FFY governs federal funding allocations and FNS reporting; it is not a

ProgramGroupEnrollments

Separation at State Level

One row exists for each sponsor enrolled in a particular program year:

Utah DOE sponsor #1247 is enrolled in NSLP SY2025-26.

It has a StatusCode (Active, Terminated, Pending)

terminated codes (why enrollment ended)

and event history.

This is the record that says a sponsor is a valid participant. It lives in **cnp_ops** because it's state-owned data.

`ProgramGroupYear` boundary and should not appear in StartDate/EndDate for school nutrition programs.

`ProgramGroupYears` is defines what a program year *is* (boundaries, program, year label). It carries no operational state. Whether a given state has opened, closed, or rolled over that year is tracked separately in `TenantProgramYearStatus` in `cnp_ops`r.

ClaimRateAmounts

Federal reimbursement rates by ProgramCode, MealType, and EligibilityCategory.

Looked up at claim calculation time using the claim date as the effective date key. INSERT annually when USDA publishes new rates. Lives in cnp_global.

ProgramRatePeriods

Rate period boundaries per program year

This table exists primarily to handle the structural difference between SFSP's rate cycle (January 1 – December 31) and the July 1 – June 30 cycle used by NSLP, SBP, and CACFP. A single effective date on `ClaimRateAmounts` cannot represent both cycles — `ProgramRatePeriods` provides the per-program boundary that tells the

TenantProgramYearStatus

Tracks each state's operational status for a program year independently. States close out a program year on their own schedule.

state A may close SY2025-26 in August while state B is still processing September claims.

The rollover job and claim submission cutoff logic read this table, not `ProgramGroupYears`, to

claim engine which rate applies to a given claim date.

A secondary use is mid-year rate adjustments. When USDA issues a revised rate partway through a program year (as happened in January 2022 for COVID relief), the current period's `EndDate` is set and a new row is inserted with the revised rate and a new `StartDate`. This is an exceptional case, not the table's primary purpose.

The table is intentionally decoupled from `ProgramGroupYears.StartDate/EndDate` — a CACFP PGY whose administrative year runs October 1–September 30 will have rate period rows starting July 1, which predates the PGY itself. This is correct by design.

determine a tenant's current year status.

cnp_ops - one per state schema

TenantProgramYearStatus table

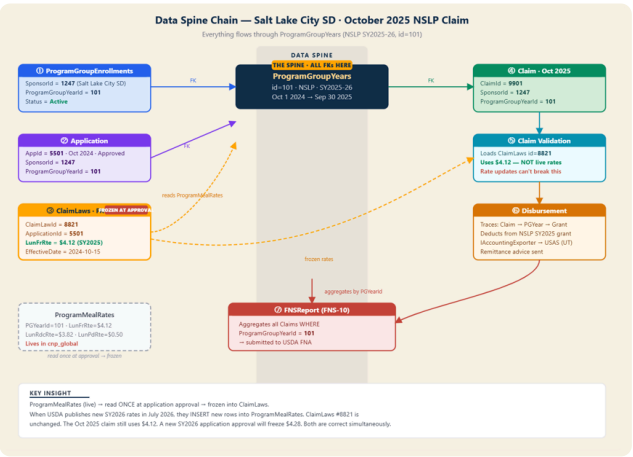
Column	Notes
StatusId	PK
ProgramGroupYearId	FK → cnp_global.ProgramGroupYears
TenantCode	Which state

StatusCode	Open | ClaimsLocked | YearEndProcessing | Closed | RolledOver
UpdatedAt	When the status changed
UpdatedBy	FK to user who made the change

The rollover job reads this table (not `ProgramGroupYears`) t

Example

Salt Lake City School District submitting their October 2025 NSLP claim.



1. The school district is enrolled in **ProgramGroupEnrollments** → NSLP SY2025-26, SponsorId = 1247, Status = Active
2. When they applied in October 2024 and got approved, the system created an **Application** with `ProgramGroupYearId → NSLP SY2025-26`
3. At application approval, `ClaimLaws.FreezeOnApproval()` read the **ProgramMealRates** rows for NSLP SY2025-26 and stored them into a ClaimLaws record linked to that application
4. When the sponsor submits the October 2025 claim, the system creates a **Claim** with `ProgramGroupYearId → NSLP SY2025-26`
5. Claim validation loads the ClaimLaws frozen at step 3, not the live ProgramMealRates table
6. When the claim is approved and locked, the **Disbursement** traces back: Disbursement → Claim → ProgramGroupYearId →

ProgramGroupYear → ProgramMealRates → knows which grant account to deduct from

7. When the **FNSReport** is generated, it aggregates all Claims for `ProgramGroupYearId = NSLP SY2025-26` for this tenant

The most important relationship is the dashed orange line: ProgramMealRates is **read once** at application approval, frozen into ClaimLaws #8821 at $4.12. From that point forward, Claims and validation load ClaimLaws — never the live rate table.

ℹ Every step in that chain joins through **ProgramGroupYears**.

THE PROBLEMS THIS DATA SPINE SOLVES

This implementation solves 4 problems:

1. **Join consistency breaks**. If Claims stores `programCode = 'NSLP'` and Applications stores `programCode = 'SNP'` and some legacy table stores `programCode = '9'`, we cannot reliably join them. ProgramGroupYears is a stable integer FK so joins are always exact.
2. **Year boundary logic becomes application code.** With loose strings, every query that spans a year boundary has to know that NSLP SY2025-26 started Oct 1, 2025. With ProgramGroupYears, that knowledge is in the database: `WHERE pgyr.StartDate <= @claimDate AND pgyr.EndDate >= @claimDate`.
3. **Rate changes add risk.** If Claims stores the rate directly, a rate correction in July affects every past claim that gets reprocessed. The spine separates the rate lookup table (`ClaimRateAmounts`, effective-dated) from the frozen eligibility snapshot applied to a specific claim (`ClaimLaws`).. The spine makes this separation structurally enforceable.
4. **Multi-program sponsors are hard to query.** A sponsor like Salt Lake City SD participates in NSLP, CACFP, and SFSP. If program is a string field on the sponsor record, we end up with

three sponsor rows or a comma-separated list. ProgramGroupEnrollments gives us a clean one-to-many: one sponsor, three enrollment rows, one per program.

Tier 1

CLAIMS & REIMBURSMENT

85 tables · Tier 1 Authoritative · Money and Enforcement

When a school district submits their monthly report of how many free, reduced, and paid meals they served, that report lives here. The system checks it against the rules that were locked in when the district applied. If something doesn't add up, the error is flagged here and the payment is held until it's resolved.

Claims is the parent record. When a sponsor submits a monthly claim, one Claims row is created. **ClaimDetails** has one row per site per claim containing the actual meal counts (Free/Reduced/Paid) plus NonCongregateTtl for non-congregate modes. **ClaimLaws** ClaimLaws is the most architecturally significant entity on the slide. It stores the eligibility snapshot extracted from the approved application at the time of approval — enrollment counts, rate tier qualifications (Severe Need eligibility, CEP ISP percentage, performance-based eligibility), and site program configuration. It does not store dollar rates. Dollar rates are looked up from ClaimRateAmounts at claim calculation time using the claim date as the effective date key. The two mechanisms work together: ClaimLaws prevents application changes from retroactively altering a sponsor's tier qualifications on already-processed claims; ClaimRateAmounts effective-dating ensures a revised prior-year claim always resolves to the rate in effect on its original claim date, not the current year's rate.. **ClaimErrors** tracks every validation failure with a BrokenInd flag — when all errors are resolved, the claim can proceed. **AcceptClaims** and **AcceptClaimSessions** handle the batch acceptance workflow where state agencies lock and approve groups of claims together. **Adjustments** handles post-approval corrections.

Claims
ClaimId PK
SponsorId FK
ProgramGroupCode
ClaimDate

ClaimDetails
ClaimDetailId PK
ClaimId FK
SiteId FK
FreeMeals

ClaimLaws

StatusCode
IsLocked

ReducedMeals
PaidMeals
NonCongregateTtl

ClaimLawId PK
ApplicationId FK
Rules JSONB
EffectiveDate

ClaimErrors
ErrorId PK
ClaimId FK
ErrorCode
SeverityInd
BrokenInd

AcceptClaims
AcceptClaimId PK
ClaimId FK
BatchDate
ApprovedAmount

Adjustments
AdjId PK
ClaimId FK
Amount
AdjType

Two DIS capabilities: auto-decline (Maine's feature flag for automatic declination of late/uncorrectable claims) and Section 4/11 split stored in ClaimLaws for FNS-10 reporting.

ACCOUNTING & FINANCE

⚠ **This section is a stub.** The accounting subdomain (grants, disbursements, repayment, FNS financial reporting, and independent audit tracking) requires its own design document before these table descriptions can serve as spec. Do not build from this section.

147 tables · Tier 1 Authoritative · financial instruments

When a state agency is ready to pay a school district for their approved meals, the payment runs through here. The federal grant that funds the payment is tracked here. If a school district was overpaid and owes money back, the repayment plan lives here. Every dollar that enters or leaves the program on the state agency's side is recorded here.

This is the financial chain: Grants fund Disbursements; FNSReports aggregate the activity; RepaymentSchedules track money owed back; and AnnualFinancialReport (Indiana + Maine reference implementation) tracks independent financial audits.

Grants
GrantId PK
ProgramGroupYearId FK
GrantAmount
FiscalYear

Disbursements
DisbursementId PK
GrantId FK
DisbursementBatchId FK
Amount

FNSReports
ReportId PK
ReportType
PeriodStart
SubmissionDate

RepaymentSchedules
ScheduleId PK
ReceivableId FK
DueDate
AmountDue

AnnualFinancialReport
AFRId PK
StateCode (IN, ME)
FiscalYear
SubmissionStatus

ELIGIBILITY & DIRECT CERTIFICATION

147 tables · Tier 1 Authoritative · eligibility determination

Before a child can receive a free or reduced-price meal, someone has to establish that they qualify. For many children, that determination happens automatically: the state matches school enrollment records against food assistance databases like SNAP and Medicaid overnight. If the system finds a match, the child is certified without anyone filling out a paper form. This section is where that matching process runs, where uncertain matches wait for a human to confirm them, and where the final list of eligible students is kept.

This is a dual-layer DC model: Core2_DirectCertificationMatches (the modern 98-table layer present in CO/IA/OH/WY but absent from Georgia) sitting alongside FreeRedcStudentEligibility (the legacy dbo layer). DirectVer Session is Indiana's unique contribution, the only DIS state that lets sponsors resolve possible DC matches themselves rather than waiting for state agency intervention. OctoberSurveyForms captures the annual building-level enrollment data used for verification sample sizes.

Core2_DirectCertMatches

FreeRedcStudentEligibility

AnnualVerificationForms
FormId PK

MatchId PK
StudentId FK
CertSource
MatchDate
EligibilityLevel

EligId PK
StudentId FK
FreeCount
ReducedCount
SchoolYear

SponsorId FK
SchoolYear
SampleCount
VerifiedCount

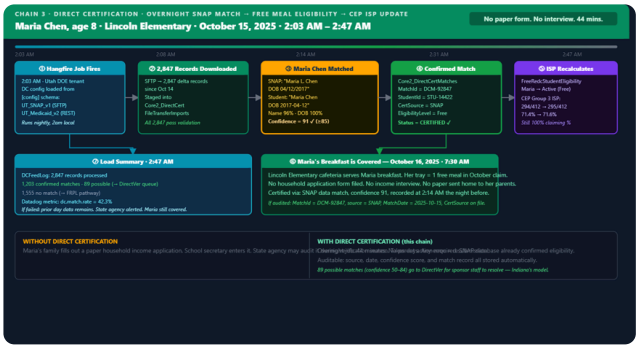
DirectVer Session (IN)
SessionId PK
RunDate
MatchCount

OctoberSurveyForms
SurveyId PK
ProgramGroupYearId FK
StateCode
EnrollmentData

EligibilityDetrerminations (cnp_ops)
DeterminationId PK
StudentId FK
ProgramGroupYearId FK
EligibilitySource
EligibilityCategory
EffectiveFrom
EffectiveTo
SourceRecordId
DeterminedAt
DeterminedBy

EligibilityQueryLog
QueryId PK
StudentId FK
QueryDate
QueriedAt
ResultCategory
DeterminationId

RequestSource



Example

A Student Is Confirmed for Free Meals Overnight

Maria Chen · Lincoln Elementary · Salt Lake City SD · October 15, 2025

Maria Chen is 8 years old. She enrolled at Lincoln Elementary in September. Her family participates in SNAP. She has not submitted a household income application. The system will certify her automatically tonight.

1. **Hangfire job fires at 2:03 AM.** The nightly DC load job starts for Utah DOE tenant. Tenant config (from cnp_global [config] schema) shows two feeds are active: UT_SNAP_v1 (SFTP) and UT_Medicaid_v2 (REST). IDcFeedAdapter resolves the SNAP adapter first.
2. **Delta records downloaded.** The adapter connects to the Utah SNAP SFTP server, downloads 2,847 records that changed since October 14. These are staged into Core2_DirectCertificationFileTransferImports. All 2,847 pass format validation (required fields present, dates parseable, no blank identifiers).
3. **Match processing runs.** The matching engine compares each staged record against the student roster for Utah DOE. The SNAP record contains: "Maria L. Chen, DOB 04/12/2017, 437 South Temple, Salt Lake City." The student roster contains: "Maria Chen, DOB 2017-04-12, 437 South

Temple, Salt Lake City, enrolled Lincoln Elementary (SiteId = 1089)." Name match: 96%. DOB match: 100%. Address match: 78% (minor formatting difference). Combined confidence score: **91**.

4. **Confirmed match. Record created.** Score ≥ 85 threshold → Confirmed Match. Core2_DirectCertificationMatches record: MatchId = DCM-92847, StudentId = STU-14422, SiteId = 1089, TenantId = ut-doe, ProgramYearId = 101 (NSLP SY2025-26), CertSource = SNAP, MatchDate = 2025-10-15, EligibilityLevel = Free. FreeRedcStudentEligibility updated: Maria is now Active, eligible for free meals.
5. **ISP recalculates for Lincoln Elementary's CEP group.** Lincoln Elementary is part of CEP Group 3 (Salt Lake City SD's west-side schools). Before Maria's match: 294 of 412 enrolled students certified = 71.4% ISP, Claiming % = 100% (already capped). Maria's match makes it 295 of 412 = 71.6% ISP — still 100% claiming %. No change in reimbursement for this group, but the record is accurate. If this had pushed a group from below 40% to above 40%, the CEPGroup's ClaimingPct field would have updated and the change would have cascaded to the next claim.
6. **Maria's meals are covered at breakfast tomorrow.** No household application form. No parent interview. No paper sent home. When Lincoln Elementary's cafeteria staff serve Maria breakfast on October 16, her tray counts as a free meal in the October claim. If a state agency auditor reviews

Lincoln Elementary next year, Maria's certification record shows the SNAP source, the match date, and the confidence score.

- **Audit trail for this determination:**
  i. `DirectCertifications` row — source, effective dates, match timestamp.
  ii. `EligibilityDeterminations` row — which rule produced the coverage decision and when.
  iii. If the application changed post-approval, `FieldChangeLog` records the delta; existing `ClaimLaws` is unaffected.
  iv. At point of service, the POS system queries eligibility as of today's date against the effective date range. The query is logged in `EligibilityQueryLog` with the result returned.

7. **Load summary logged at 2:47 AM.** DCFeedLog record created: 2,847 records processed, 1,203 confirmed matches, 89 possible matches (confidence 50–84, queued for DirectVer resolution by sponsor staff), 1,555 no match, 0 errors. Datadog metric emitted: dc.match.rate = 42.3% for Utah DOE. If the load had failed, a prior day's data would remain active and an alert would fire to the state agency contact — Maria would still be covered under the previous certification.

Without this overnight process, Maria's family would need to fill out a paper household income application. The school's secretary would enter it into the system. A state agency staff member

might audit it during verification season. The process would take days. The automated match takes 44 minutes for the entire state.

COMPLIANCE & ADMINISTRATIVE REVIEW

~375 tables · Tier 1 Authoritative · enforcement and fiscal accountability

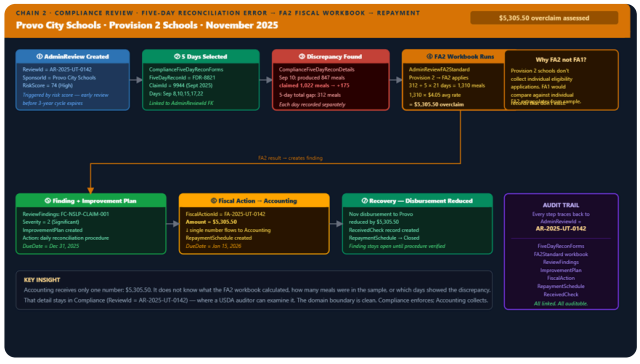
When a state agency visits a school to check whether meals are being served correctly, every piece of evidence they collect is recorded here. If they find the school overclaimed meals, the financial penalty is calculated here too. If the situation is serious enough to remove a sponsor from the program entirely, that legal record lives in its own protected space.

The mental model of compliance review is simple: reviewer visits sponsor, records pass/fail, done. The reality is much more complex. The USDA requires state agencies to collect extremely specific evidence for each type of finding, and each finding type has its own data structure. A meal observation form for a CACFP center has different fields than a meal observation form for a DCH provider. A five-day food production reconciliation has different fields than a milk audit. Each of those requires its own detail table, its own error table, its own status table, and its own summary table. Multiply that across all program types and all finding categories and 375 tables is where you land.

Example

A Five-Day Reconciliation Error Leads to a Repayment

Provo City Schools · Utah · November 2025 compliance review



Provo City Schools operates 41 NSLP sites, including several Provision 2 schools. Their risk score is 74 (High) — the last review was January 2024, and there are 3 open findings from that visit. The 3-year cycle would allow waiting until 2027, but the risk score triggers an early review.

1. **Review scheduled.** State agency creates an AdminReview record: ReviewId = AR-2025-UT-0142, SponsorId = Provo City Schools, ReviewType = On-Site

Administrative Review, StatusCode = Scheduled. The reviewer is assigned based on program specialization and risk queue.

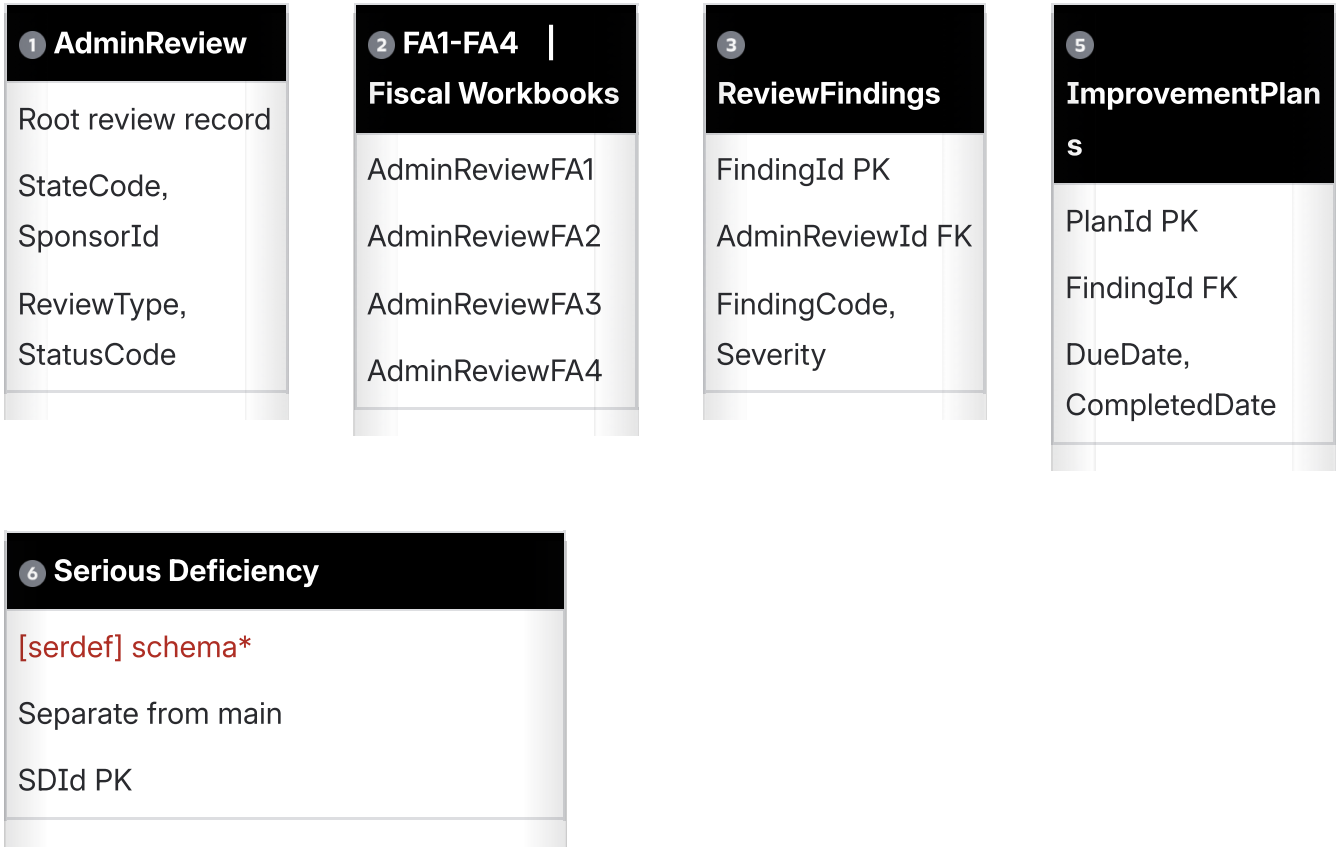
2. **Reviewer selects 5 days from the September 2025 NSLP claim.** Provo's September claim (ClaimId = 9944) shows 22,840 meals served. Reviewer creates a ComplianceFiveDayReconForms record: FiveDayReconId = FDR-8821, linked to AdminReviewId = AR-2025-UT-0142 and ClaimId = 9944. She selects September 8, 10, 15, 17, and 22 as the five sample days.
3. **Day 2 discrepancy found.** On September 10, food production records show 847 meals prepared and served. The claim shows 1,022 meals for that day. ComplianceFiveDayReconDetails records the gap: 175 meals. She works through all five days. Total gap across the sample: 312 meals over 5 days.
4. **FA2 fiscal workbook runs.** Provo operates under Provision 2, so the standard FA1 formula doesn't apply — you can't compare against individual eligibility records for a Provision school. The FA2 variant is used instead. AdminReviewFA2Standard record created. Calculation: 312 meals ÷ 5 sampled days × 21 operating days in September = 1,310 estimated overclaimed meals. At the blended reimbursement rate for Provo's meal mix: 1,310 × $4.05 average = **$5,305.50 overclaim assessed.**
5. **Finding and improvement plan created.** ReviewFindings record: FindingCode = FC-NSLP-CLAIM-001, Severity = 2

(significant). ImprovementPlan created: Provo must implement a daily reconciliation procedure between kitchen production counts and point-of-service counts. DueDate = December 31, 2025.

6. **Fiscal action flows to Accounting.** Claim records identified → proposed → formal notice + appeal deadline → final determination → repayment. You don't need to add the full swimlane — a concise paragraph describing the status progression (`Proposed → Issued → Appealed → Final → RepaymentSchedule created`)FiscalActionId = FA-2025-UT-0142, Amount = $5,305.50. This single number moves to the Accounting domain: RepaymentSchedule created with DueDate = January 15, 2026. Accounting does not know or care how the $5,305.50 was calculated, it only receives the assessed amount.
    a. Audit trail: `AdminReviews` row establishes the review that produced the finding. `FiscalActions.ReviewId` links back to it. The specific claims that were overclaimed are linked via `FiscalActionClaims (FiscalActionId, ClaimId)`. The status history (Proposed → Issued → Final) is tracked in `EntitySnapshots`. `RepaymentSchedules.TriggerReferenceId` links forward to the fiscal action that created it. The full chain can be resconstructed from FKs .

7. Provo does not dispute. The state agency's November disbursement to Provo is reduced by $5,305.50. A ReceivedCheck record is created when the deduction is applied, and the RepaymentSchedule is closed. The finding remains open until Provo demonstrates their new reconciliation procedure is in place.

> Every step from the five-day worksheet through the FA2 calculation to the repayment deduction traces back to AdminReviewId = AR-2025-UT-0142. A USDA auditor examining this review two years later can follow the complete chain: what days were sampled, what the discrepancy was, which fiscal formula was applied, how the amount was calculated, and when the money was recovered



*Serious Deficiency

`SeriousDeficiency` lives in a separate `[serdef]` schema for regulatory reasons. When USDA's regulations require a state agency to initiate serious deficiency proceedings, the documentation of that proceeding must be legally defensible in an appeal. Placing it in a separate schema creates a clear audit boundary. No other domain's queries can accidentally modify SerDef records. The schema isolationsupports the regulatory requirement that SerDef records be immutable once issued.

The SerDef data model includes:

- `SeriousDeficiency` the core record
- `SeriousDeficiencyTriggers` what finding triggered the proceeding (there are specific regulatory categories that mandate a SerDef)
- `SeriousDeficiencyUnits` which of the sponsor's sites or homes are implicated
- `SeriousDeficiencyAppealTypes` the sponsor's options for contesting
- `TerminationEvents` If the proceeding results in termination from the program

A SerDef that's resolved without termination is closed; one that results in termination is permanently recorded in `TerminationEvents` and affects whether the sponsor can reapply in the future.

FA1-FA4 fiscal workbook variants

When a review finds overclaiming, the state agency must calculate how much money the sponsor owes back. USDA prescribes specific formulas depending on what type of school the sponsor operates and how they claim meals. The schema reflects four distinct fiscal workbook calculations:

FA1	FA2	FA3	FA4
This is the standard fiscal action formula for regular NSLP/SBP schools. It calculates the overclaim amount from the difference between what was	This is the Provision 2 and Provision 3 variant. Schools operating under Provision 2 or 3 don't collect individual meal applications — they claim based on historical	follows the same Provision 2/3 split but applies a different disallowance methodology. In some cases USDA requires the agency to apply the fiscal action at	The most specific variant — it applies when the overclaim involves non-creditable meals (meals that were claimed but didn't meet the meal pattern requirements).

claimed and what should have been claimed, applied across the full school year. The tables

`AdminReviewFA1Summary`

`AdminReviewFA1Meals`

`AdminReviewFA1Errors`

capture the inputs, calculated amounts, and any errors in the workbook.

percentages established in a base year. Fiscal action for these schools requires a different calculation because the overclaim can't be measured against individual eligibility records. Hence,

`AdminReviewFA2Standard`

`AdminReviewFA2Provision2`

`AdminReviewFA2Provision3`

`AdminReviewFA2Summary`

three sub-variants depending on exactly which provision applies.

the site level rather than the sponsor level. The tables

`AdminReviewFA3Standard`

`AdminReviewFA3Provision2`

`AdminReviewFA3Provision3`

mirror FA2's structure but with different calculation rules.

`AdminReviewFA4Details`

`AdminReviewFA4Forms`

capture the specific meals that were disallowed and the resulting fiscal impact.

Recalculation Layer

Beyond the four variants, there's a recalculation layer:

`AdminReviewFARecalc_NSLP`, `AdminReviewFARecalc_NSLPDetails`, `AdminReviewFARecalc_P2NSLP`, `AdminReviewFARecalc_SBP` and others. When a sponsor disputes a fiscal action or provides additional documentation, the state agency may need to recalculate. These tables preserve the original calculation alongside the revised calculation so both are auditable.

The recalc tables are why "fiscal action" is under Compliance rather than Accounting. The *calculation of what's owed* is an enforcement function tied to the review record. The *collection of what's owed* (Repayment Schedules, Receivables, ReceivedChecks) lives in Accounting.

Compliance worksheets

The `Compliance*` table cluster is where the granular on-site evidence lives. Each worksheet type corresponds to a specific thing a reviewer examines during the visit:

Meals Served	Financial Records	Eligibility	Meal Pattern Compliance
What the reviewer physically observes being served	Reconciling what was purchased/produced against what was claimed	Are the right students claiming free and reduced meals?	Meals served that don't meet federal requirements

Meal Observation
Center-based: components, portions, consumption
`ComplianceMealObservationForms`
`ComplianceMealObservationDCHDetails`
	Five-Day Food Production Recon
	Most common finding source — production records vs. claim counts
	`ComplianceFiveDayReconForms`
	`ComplianceFiveDayReconDetails`
		Enrollment & Eligibility
		Samples F/R eligibility determinations and documentation
		`ComplianceEnrollmentEligibilityForms`
		`ComplianceEnrollmentEligibilityDetails`
		`ComplianceEnrollmentEligibilityEnrollmentDetails`
		`ComplianceEnrollmentEligib`
			Menu Disallowance
			Wrong components or portions — feeds directly into FA4 calculation
			`ComplianceMenuDisallowanceForms`
			`ComplianceMenuDisallowanceTotals`
			`ComplianceMenuDisallowanceDCHForms`
			`ComplianceMenuDisallowanceDCHTotals`

Infant Meal Observation (DCH) DCH only — infant formula & breast milk requirements `ComplianceInfantMealObserv`	**Milk Audit (NSLP/SBP)** Purchase records vs. milk served; shortage triggers separate fiscal review		

`ationDCHDetails`	`ComplianceMilkAuditForms` `ComplianceMilkAuditMealDetails` `ComplianceMilkAuditShortageDetails`	`ilityDCHForms`	**Non-Creditable Meals** Ineligible participant or wrong service time. Distinct from disallowance `ComplianceNonCreditableMealsForms` `ComplianceNonCreditableMealsDetails`
Camp Session (SFSP) Field trips, multiple locations, irregular times `ComplianceCampSessionForms` `ComplianceCampSessionDetails`	**Milk Quantity Assessment (NSLP/SBP)** Volume verification — reviewer checks assessment methodology `ComplianceMilkQuantityAssessmentForms`	**Classroom Data (NSLP/SBP)** Building-level enrollment vs. meal counts claimed `ComplianceClassRoomDataForms` `ComplianceClassRoomDataDetails`	
Snack Calculator (SFSP) Attendance-based snack entitlement calculation `ComplianceSnackCalculatorDetails`			

School Reviews

SchoolReviews is an SNP-specific compliance track that runs alongside the general administrative review but examines schools at a different level of granularity. A sponsor like a school district might operate 40 schools. The administrative review examines the sponsor's systems and controls. The school review examines individual school sites for the fiscal action calculation when overclaiming is identified there rather than at the sponsor level.

- `SchoolReviews` → `SchoolReviewDetails` → `SchoolReviewDetailsDays` captures meal counts at the daily level for each school examined.
- `SchoolReviewFADetails` is the school-level fiscal action calculation when the overclaim is traced to a specific school rather than the sponsor overall, the disallowance must be calculated school by school.
- `SchoolReviewMissingItems` tracks documentation gaps at each school

- `SchoolReviewModHistory` is the modification history. School review data can be corrected after initial submission, we need to document who changed what and when.

Risk scoring and how it feeds the 3-year cycle

`RiskScores` is not a single computed number. It's a multi-factor calculation that the system runs periodically for every sponsor. The inputs typically include:

- time since last review
- any open findings from prior reviews
- claim error rate over the past year
- poverty level of the service area
- any compliance history with FNA
- whether the sponsor has been flagged by another agency.

The output is a risk level (Low / Medium / High) and a computed score. High-risk sponsors are reviewed annually rather than once every three years.

`ReviewCycles` (in the Reviews domain) is driven by `RiskScores`. The risk score determines both the frequency of review and the prioritization within the review queue. State agencies with limited reviewer capacity use risk scores to decide which sponsors get reviewed this year versus next year. Without risk scoring, a state agency would either review sponsors randomly or in alphabetical order, both of which miss the sponsors most likely to have problems.

Procurement Review (FSMC)

`ProcurementReviews` is a compliance track that applies to all sponsors under 2 CFR 200 and 7 CFR 210.21 — not just those using a Food Service Management Company. FSMC is an additional conditional layer on top of the baseline, not the gate for the review itself. All sponsors are subject to the general procurement section:

- Competitive bidding processes, solicitation methods, small purchase thresholds, and general procurement compliance under 2 CFR 200 FSMC sponsors only are subject to additional sections:
- Contract management, management fee reconciliation, USDA Foods pass-through, fee caps (sponsor-level)
- FSMC contract amendments, USDA Foods management at the site level, contract performance (site-level)

A procurement review examines whether the sponsor followed required procurement procedures. For FSMC sponsors, it also examines the FSMC contract terms and compliance.

The data model for procurement review includes:

- `AdminReviewProcurementAuthority` who authorized the procurement decision
- `AdminReviewProcurementCodes` the specific regulatory requirements being examined
- `AdminReviewProcurementVendors` the vendors who bid and the selection rationale
- `AdminReviewProcurementSmallPurchase` and `AdminReviewProcurementGroupPurchase` different procurement methods that have different requirements (apply to all sponsors)
- `AdminReviewProcurementCashInLieu` for SFSP sponsors who provide cash instead of commodities

Procurement review can happen during a regular administrative review or as a standalone review triggered by a complaint or by the sponsor entering into a new FSMC contract. This is why `ProcurementReviews` has a FK to `AdminReviewId` that can be null. A standalone procurement review exists without a parent administrative review.

Fiscal actions moved to Compliance in this model. A fiscal action is the consequence of a review finding of overclaiming. It doesn't represent money the state agency is owed in the same way a grant disbursement represents money the state agency owes a sponsor. It represents the quantified consequence of an enforcement action. The repayment schedule that flows from a fiscal action, and the actual recovery of funds, belongs in Accounting because that's the financial instrument. But the calculation of *how much* is owed, and the legal record of *why*, belongs in Compliance where the review lives.

If fiscal actions live in Accounting, we create a circular dependency: the Accounting domain needs to know about Review findings to generate fiscal actions, which means Accounting needs to read from Compliance. With fiscal actions in Compliance, Accounting only needs to receive the final assessed amount and create a Repayment Schedule against it. The domain boundary is clean.

FiscalActions
FAId PK
ReviewId FK (Admin Reviews)
FAType
Assessment Amount
StatusCode

FiscalActionClaims
FiscalActionClaimId PK
FAId FK (Fiscal Actions)
ClaimID FK (Claims)
OverclaimedAmount

AnnualAuditForms
AuditFormId PK
SponsorId FK
FiscalYear
AuditType
SubmissionDate

AppealDeadline
IssuedAt
IssuedBy FK (Users)

RiskScores
RiskId PK
SponsorId FK
RiskLevel
ScoreDate
ComputedScore

SchoolReviews
SchoolReviewId PK
AdminReviewId FK
SNP-Specific Track
SchoolId FK

ProcurementReviews
ProcReviewId PK
SponsorId FK
ContractType
CompetitiveProcess

Tier 2 | Core Supporting Domains

Operational backbone: sponsors, applications, access, and configuration

Sponsor & Site Registry

Every organization that participates in the program (school districts, daycare centers, summer camps) is registered here. Every location where meals are actually served is registered here too. This is the authoritative list of who is in the program and where they operate. When a new organization wants to join, they start here. When a site closes, that change is recorded here.

This holds the master records of who participates. Sites now carry Latitude and Longitude, generalizing the Missouri ArcGIS pattern to all states rather than keeping it Missouri-only.

Applications & Agreements

Every year, each participating organization has to renew their agreement to follow program rules. That renewal process, including the forms, the attachments, the approval workflow, lives here. Before a new organization can participate, they go through a pre-screening process that's also tracked here. The forms change slightly each year, and this section keeps track of which version of each form was in effect when an organization applied.

This the largest Tier 2 domain at ~173 tables, covering the full application lifecycle including PotentialSponsor (a DIS capability brought forward that tracks prospects before formal application), the AnnualForms generalized framework, FSMC contracts, and VCA Checklists.

Identity & Access

Every person who logs into the system is governed by what's stored here. A state agency reviewer can see all sponsors in their state; a school district employee sees only their own district's data. These boundaries are enforced at the service layer on every request.

This section contains the RBAC tables that replaced the legacy CnpObjects ACL. Rather than 5,082 individual permission rows, the new model uses approximately 180 named permissions organized into roles. Per-state variation is handled through `TenantRoleOverrides` rather than separate permission definitions per state. The core tables are: `PermissionCatalog`, `Roles`, `RolePermissions`, `TenantRoleOverrides`, `UserRoleAssignments`, and `SponsorUserAccess`. Full design in CNP Rights Model.

Configuration & Tenant

USDA publishes new reimbursement rates every July. Rather than requiring a software update every year, those rates are stored in `ClaimRateAmounts` with an effective date. When July 1 arrives, the new rates are available for claim calculation. Applications approved after July 1 will have a ClaimLaws eligibility snapshot that captures the new school year's tier qualifications. Claims resolve dollar rates from `ClaimRateAmounts` using the claim date as the effective date key. Prior-year claims always return prior-year rates regardless of when they are revised or reprocessed. State-specific settings that don't involve federal rules (workflow preferences, notification templates, local thresholds) live here too.

`ClaimRateAmounts` sits between meals served and the federal funding available to pay for them. Given a `ProgramCode`, `MealType`, and `EligibilityCategory`, it returns the reimbursement rate the federal government applies to that meal. Annual rate updates are INSERT statements here, not code deployments. Lives in `cnp_global` — no tenant scope, because federal rates are the same across all states.

Sponsor & Site Registry ~121 tables	Applications & Agreements ~173 tables	Identity & Access (IAM) ~79 tables	Configuration & Tenant ~35 tables
Customers / Sponsors	Applications	PermissionCatalog	ProcReviewId PK
Sites (Lat/Long)	SponsorApplications	Roles	SponsorId FK
Providers	SiteApplications	RolePermissions	ContractType
Addresses	AppPackets	TenantRoleOverrides	CompetitiveProcess

BoardMembers

Holds the master records of who participates. *Latitude/Longitude on all Sites — generalizes Missouri ArcGIS pattern.*

AnnualForms
FsmcContracts
PotentialSponsor
VCAChecklists

C**overs the full application lifecycle.** *PotentialSponsor added - DIS capability brought forward.*

UserRoleAssignments
SponsorUserAccess

Tier 3

PROGRAM DOMAINS (SNP, CACFP, DCH, SFSP, SSFO)

Specialist domains, each with its own application/claim/compliance stack

Program Domains (SNP, CACFP, DCH, SFSP, SSFO) represent one federal program with its own application/claim/compliance stack.

School Nutrition Programs (SNP)

The National School Lunch Program is the flagship program that feeds the most children in the most schools every day. This section covers the rules, forms, and calculations specific to school meal programs: which schools qualify for enhanced funding, how summer meal service works, how meal pattern compliance is tracked, and how schools that serve all students for free under the Community Eligibility Provision calculate their reimbursement.

It includes OctoberSurveyForms and ProvisionRatios (Provision 2 and 3 provisional percentage tables).

Child & Adult Care Food Program (CACFP)

Daycare centers, after-school programs, and adult care facilities operate under a different set of rules than K-12 schools. Children in these settings eat different meal patterns at different times, and the providers serving them range from large nonprofit institutions to individual home-based providers. This section covers the forms, claims, and compliance requirements for all of them, including the annual self-assessment that every sponsoring organization must complete to demonstrate they're capable of running the program.

This includes VCAChecklists, the 16-table self-assessment that gates application approval.

Day Care Home (DCH)

Home-based childcare providers — someone running a licensed daycare out of their own house — are treated differently from institutional daycare centers. They participate through a sponsoring organization that monitors them, and they're reimbursed at rates that depend on whether the provider and neighborhood meet low-income thresholds. This section tracks every home-based provider, their applications, their monthly meal claims, and which reimbursement tier they fall into.

This is a distinct provider-based model where individual home-based providers claim meals under a sponsoring organization. It does not share the site model used by CACFP centers.

Summer Food Service Program (SFSP)

When school is out and cafeterias are closed, children in lower-income communities lose access to the free meals they depend on during the school year. The Summer Food Service Program fills that gap by funding meals at summer camps, parks, libraries, and community centers. This section covers the sites that serve summer meals, how sites can distribute meals as pickup or grab-and-go when children can't gather in one place, and how those summer operations are funded and monitored.

This includes NonCongregateRequest with the three-mode enforcement model (congregate only, non-congregate only, mixed).

Seamless Summer Food Option (SSFO)

Some schools that already operate the National School Lunch Program want to continue serving meals through the summer without switching to different rules and paperwork. The Seamless Summer Food Option lets them use school-year administrative procedures but summer funding. This section tracks those schools separately from SFSP so that the two programs' different rules don't get mixed together.

SSFO was added as a first-class program domain based on the architect's recommendation; it has ~16 dedicated tables with distinct ClaimLaws from NSLP, preventing SSFO quirks from bleeding into NSLP claim validation.

Fresh Fruit and Vegetable Program (FFVP)

A USDA grant that gives selected elementary schools a dollar allocation to purchase and serve fresh fruits and vegetables as snacks during the school day. It is not a meal reimbursement program — there's no per-meal rate and no ClaimLaws. Schools apply for a per-student budget allocation, spend it on produce, and report expenditures back to the state.

The financial instrument is completely different from every other program. Instead of "meals served × reimbursement rate = payment," the model is "approved allocation → expenditure tracking → unexpended balance." A school doesn't submit a monthly meal count claim; they submit an expenditure report showing what they spent on produce.FFVPSiteInvitations

Community Eligibility Provision (CEP)

An election within NSLP/SBP that lets high-poverty schools serve all students free meals without collecting individual household income applications. A school qualifies if its Identified Student Percentage (ISP) — the ratio of directly-certified students to total enrollment — is 40% or higher. Once elected, the school is reimbursed at a blended rate: (ISP × 1.6) × free meal rate + remainder × paid meal rate. The multiplier of 1.6 is set by federal regulation and cannot be configured.

Changes how reimbursement is calculated for SNP/NSLPclaims. The ISP calculation, the group management (schools can pool together to hit the 40% threshold), and the annual election process all have their own data structures that would clutter the SNP tables if embedded there.

SNP/NSLP ~48 tables
•NslpSponsorApps
•NslpSiteApps
•MealPatternCompliance
•OctoberSurveyForms
•ProvisionRatios
•FreshFruitsLineItems

FreshFruits is an FFVP sub-component.

CACFP ~58 tables
•CacfpSponsorApps
•CacfpSiteApps
•VCAChecklists
•InstitutionMgmtPlans
•AcqrReports

VCAChecklists shared with Applications domain. AcqrReports for procurement.

DCH ~42 tables
•ProviderApplications
•ProviderClaims
•DchManagementPlans

Day Care Home — distinct provider-based model separate from sponsor/site.

SFSP ~33 tables
•SfspSponsorApps
•SfspSiteApps
•NonCongregateRequest
SummerMandates

NonCongregateMealRequest — 3 modes enforced (virtual, grab-and-go, in-person).

SSFO table ~16
SSFOSponsorApp
SSFOSiteApp
SSFOClaimLaws

FFVP
FFVPSiteInvitations
FFVPApplications
FFVPAllocations
FFVPExpenditureReports
FFVPLineItems
FFVPBalances

CEP
NslpSponsorApps
NslpSiteApps
MealPatternCompliance
OctoberSurveyForms
ProvisionRatios
FreshFruitsLineItems

Seamless Summer Food Option — minimal domain extending SNP summer rules.

SPECIALIST MODULES

VCA Checklist Before a childcare sponsoring organization can participate in CACFP, it has to demonstrate it's actually capable of running the program responsibly: financially solvent, operationally capable, and accountable for the homes or centers under its umbrella. That annual self-assessment, scored against a standard federal rubric, is what lives here. A sponsor that scores below the passing threshold cannot have its application approved until the gaps are addressed.

Budget Module Some program types require sponsors to submit a detailed budget alongside their application by projecting their food costs, labor costs, and administrative expenses for the year. This section stores those budget submissions, the line-item breakdowns, and the approval status. It's the financial accountability layer that lets state agencies verify a sponsor is planning to spend program funds appropriately before approving their participation.

Waivers & Approvals Sometimes a sponsor needs to do something the standard rules don't permit. Each exception requires written approval from the state agency or USDA before it happens. This section tracks those requests, the approvals, and the outcomes — so there's a clear record that exceptions were properly authorized.

*Without this domain, exceptions disappear into email inboxes. A USDA auditor asking "was this sole-source contrac*t approved before the sponsor signed it?" should have a single place to go for the answer, not a search through three years of email.

VCA Checklist ~16 tables
•VCAChecklists
•VCAScoringTools
•VCAChecklistItems
•VCAChecklistSummaries

Budget Module ~19 tables
•Budgets
•BudgetDetails
•BudgetItems
•BudgetLaborDetails
BudgetMealWorksheets

Waivers ~7 tables
WaiverItems
WrittenApprovalRequests
CapitalExpenditureRequests
ApprovalDecisions
ProgramWaivers
WaiverSponsorExtensions
WaiverOutcomes

Training & Professional Standards

USDA requires that directors and staff who manage school nutrition programs complete a minimum number of training hours each year. This section tracks which courses exist in the federal catalog, which employees have completed which courses, and whether each organization's staff collectively meet the annual requirement. Hawaii's implementation can be used as the reference design when this module is rolled out to all states.

Trainings
TrainingId PK
SponsorId FK
TrainingDate
TrainingType

TrainingCourses
CourseId PK
CourseName
CourseCode
-- cnp_global

TrainingDetails
DetailId PK
TrainingId FK
AttendeeId FK
CompletionDate

ProfStandardEvents
EventId PK
TrainingId FK
StandardCode
Hours

Food Distribution

Separately from cash reimbursements, USDA distributes actual commodity foods — bulk canned goods, frozen protein, dairy products — to schools and institutions. Managing what was ordered, what was delivered, what's sitting in a warehouse, what was diverted or donated, and what the annual commodity survey shows, all lives here. This section cannot be cleanly

separated from the rest of the system without significant work, which is why a decision about whether it stays in scope needs to be made before migration planning begins.

128 tables embedded in shared dbo schema. Cannot be cleanly separated without schema surgery. Migration planning is blocked until scope is decided.

Tier 4

Cross-cutting infrastructure shared by all program domains

Reporting & Analytics (→ Snowflake) · Audit & Observability · Notifications · Reference Data & Transparency · LaunchDarkly

Every month, every state agency submits federal reports to USDA proving that program funds were spent correctly, that the right number of children were served, and that reimbursements match what was claimed. Generating those reports from a live operational database would mean running complex multi-year aggregations against the same tables that state agency staff use to approve claims in real time — a query that takes 45 seconds to build a federal summary would block a claim approval for 45 seconds. This domain solves that by maintaining a separate read-optimized copy of all operational data in Snowflake. The operational database records transactions; Snowflake answers questions about them. Nothing in this domain writes back to any other domain. It consumes facts; it never influences behavior.

Tier 4 is a different read/write contract: it only reads. The consequence of this separation is that the reporting infrastructure can be rebuilt, reconfigured, or replaced without touching a single table in the operational database. The operational system doesn't know Snowflake exists. The reporting system doesn't need to know how a claim was approved, it only needs the final approved state.

This is also why federal report restatements are safe. When a state needs to resubmit a corrected FNS-777 for a prior year, the Snowflake copy reflects the current state of the operational database including any post-approval adjustments. FNSReportLineItems stores what was submitted previously so the delta is auditable. The operational ClaimLaws records are untouched.

Documents & Attachments Every form submission, every approval, and every review generates documents — PDFs, spreadsheets, uploaded certificates. This section stores them. It also generates a snapshot — a frozen PDF of exactly what a form looked like at the moment it was submitted or approved — so that months later, anyone can see not just the current state of a record but what it looked like at every significant point in its lifecycle.

Communications & Emails State agencies communicate with sponsors regularly — deadline reminders, approval notices, requests for additional information. The templates for those communications live here, along with the record of what was sent, to whom, and when. Some states maintain email distribution lists for bulk outreach to all sponsors of a particular program type; that capability lives here too."

FTP & Integration The platform exchanges data with outside systems — state benefit agencies for Direct Certification, USDA for commodity orders, state financial systems for payment processing. The configuration for those data connections, the log of files transferred, and the record of what succeeded and what failed all live here. Several of these integrations exist today as separate applications in the LINQ codebase; those applications are architecturally sound but have not received recent maintenance and will need to be rebuilt, not simply migrated."

Workflow & Tasks Certain processes in the platform happen automatically in the background — the nightly job that checks for new benefit matches, the annual job that rolls every application forward into a new program year at midnight on October 1, the scheduled alerts that fire when a claim has been sitting unreviewed too long. This section manages the scheduling and execution of those background jobs, the task lists assigned to state agency staff, and the calendar of review deadlines.

Reporting & Analytics Every FNS report submitted to USDA — the monthly participation data, the financial status report, the annual program summary — is generated from here. Unlike the operational database, which is optimized for recording transactions, this section reads from a separate analytics warehouse (Snowflake) that is optimized for summarizing large amounts of historical data. Reports run against Snowflake instead of the main database so that generating a complex federal report never slows down a state agency employee trying to approve a claim.

Documents & Attachments ~33 tables
•DocumentLib
•FormSnapshot (PDF → S3)
•Attachments

FormSnapshot triggered on every status change.

Communications & Email ~75 tables
•EmailTemplates
•EmailDistributions
•MessageBoards
•EmailListManagement (ME, UT)

FTP & Integration ~31 tables
•FtpServers
•FtpFiles
•ProviderLicenseSync (UT)
•ApiAuthenticationTokens

Stale repos: CnpCepApi, UTDirectCertAPI,

Immutable audit trail per lifecycle event.	*EmailListManagement is state-specific (ME, UT). Template engine is shared.*	*UTLicenseSynchronizer, MOArcGisAPI*

Workflow & Tasks ~25 tables
•CnpTasks
•CnpAlerts
•CalendarEvents
•Hangfire jobs

Hangfire for background job scheduling. CnpAlerts for user-facing notifications.

Reporting & Analytics ~28 tables
CdcPipelineStatus
SnowflakeRefreshLogs
ReportDefinitions
ReportSchedules
ReportRuns
ReportSubmissions
FNSReportLineItems
StateDashboardConfigs
SavedReportFilters

The CDC Pipeline (Aurora → Snowflake)

AWS Database Migration Service runs a continuous Change Data Capture feed from every cnp_ops Aurora instance to Snowflake. Every INSERT, UPDATE, and DELETE that happens in any state's operational database is streamed to Snowflake within minutes. The pipeline architecture:

Layer	What it does
AWS DMS CDC agents	One agent per cnp_ops Aurora instance — captures the binary log and streams changes

S3 staging bucket	Raw change events land here before Snowflake ingestion
Snowpipe	Snowflake's auto-ingest mechanism picks up files from S3 continuously
cnp_analytics schema	The landing zone in Snowflake — mirrors the operational schema with added metadata columns (cdc_timestamp, source_state, operation_type)
Reporting views	Curated views over cnp_analytics that the report generators query — these are the stable API that FNS reports and dashboards depend on

cnp_global tables (rates, program definitions, lookup codes) are replicated to Snowflake on a nightly full refresh rather than CDC, since they change infrequently and the volume is small.

Federal reports this domain generates

FNS-10 — School Program Operations Report. Participation data: meal counts by program (NSLP, SBP, CACFP), meal type, and eligibility category (Free, Reduced, Paid), broken down by sponsor and site. Monthly submission to USDA. Pulls from `Claims`, `ClaimDetails`, `Sites`, `Sponsors`.

FNS-777 — Financial Status Report The monthly financial accounting report submitted to USDA. Shows grant drawdowns, expenditures, and unexpended balances. Section 4/11 split for states with both federal and state-funded meal programs (Utah is the reference implementation). Pulls from cnp_analytics views over Grants, Disbursements, and RepaymentSchedules

FNS-44 — Financial Status Report. Report of the Child and Adult Care Food Program. Participation data for CACFP: meal counts by institution type (centers, day care homes, adult care), meal type, and eligibility category. CACFP equivalent of FNS-10. Monthly submission to USDA. Pulls from CACFP claims, sites, and provider data. It's the CACFP equivalent of FNS-10.

FNS-418 — Summer Food Service Program Participation Summer-only, submitted at program close. Aggregates SFSP meals by site type (open, enrolled, camp) and service mode (congregate, non-congregate).

State-level dashboards (not FNS submissions):

- Claim pipeline status: claims submitted vs. approved vs. pending vs. held for error, by program, current month
- DC match rate by school and sponsor: percentage of enrolled students with active direct certifications
- Review cycle tracker: which sponsors are due for review, overdue, scheduled, in progress
- Risk score distribution: Low/Medium/High across the sponsor portfolio
- Annual rate effective dates: when the next ClaimRateAmounts INSERT is expected and which states are pending

Audit & Observability

~18 tables · Tier 4 Observational · append-only audit infrastructure

Every action a user takes, every field they change, and every status transition a record passes through is captured here. This domain never influences program behavior — no claim decision, no compliance outcome, and no eligibility determination reads from it. It exists entirely to answer questions after the fact: what happened, who did it, when, and what did the record look like at each moment in its lifecycle.

The domain addresses the compliance requirement from the WIP CNP Platform Replacement spec: historical compliance states must be queryable at any point in time without manual reconstruction. A USDA auditor examining a review from two years ago should be able to ask the system "what did this application look like at the moment of approval?" and receive a structured, query-ready answer — not be handed a PDF to read manually.

The domain has three tables with distinct but complementary responsibilities. AuditLogs captures who touched what and when — a lightweight record of every action. EntitySnapshots captures a full structured snapshot of a record's key fields at each status transition — the machine-queryable version of what FormSnapshot captures as a PDF. FieldChangeLog captures every individual field edit, including changes that happen between status transitions and would otherwise be invisible to the snapshot record.

Relationship to FormSnapshot

FormSnapshot (in Documents & Attachments) and EntitySnapshots are triggered by the same event — a status change — but serve different audiences. FormSnapshot renders a PDF to S3 for human review and legal defensibility. EntitySnapshots stores JSONB to the database for machine queries and report generation. An auditor opening a case file uses FormSnapshot. The

system generating a compliance summary report uses EntitySnapshots. Both are needed; neither replaces the other.

How the three audit tables work together

Table	What it captures	Format	Use case
FormSnapshot	Full form state at each status transition	PDF → S3	Human review, legal defensibility
EntitySnapshots	Key fields at each status transition	JSONB (queryable)	System time-travel, compliance queries, report generation
FieldChangeLog	Every individual field edit	Text rows	Full data change audit, intra-status edits

Together these three tables satisfy all five audit requirements from the WIP spec without event sourcing: 100% user action capture, 100% data change capture, 100% workflow transition capture, point-in-time compliance queryability, and no

All three tables are append-only

No row in AuditLogs, EntitySnapshots, or FieldChangeLog is ever updated or deleted in normal operation. The audit trail is only meaningful if it cannot be altered after the fact. Deletion of audit records requires a separate privileged data retention process outside normal application code, triggered only for regulatory compliance (e.g., GDPR deletion requests for specific individuals).

Table	**Key columns**	**What it captures**	**Query pattern**
AuditLogs	AuditId PK, TenantCode FK, EntityType, EntityId FK, ChangeType, ChangedBy FK, ChangeDate, SessionId	who did what? Every user action and system event, a lightweight record of who touched what and when	"Show all actions on ClaimId 9944 in the last 30 days"

EntitySnapshots	SnapshotId PK, TenantCode FK, EntityType, EntityId FK, StatusCode, SnapshotDate, DataJson JSONB, TriggeredBy FK	what did this record look like at time? Full structured snapshot of a record's key fields at each status transition. Machine-queryable companion to FormSnapshot.	“what was the enrollment count on this application at the time of approval?”
FieldChangeLog	ChangeId PK, TenantCode FK, EntityType, EntityId FK, FieldName, OldValue, NewValue, ChangedBy FK, ChangedAt, ChangeSource (UI/API/System)	what specifically changed between these two states? Every individual field edit, including intra-status changes between snapshots	"Show every change to SponsorId 1247's address in the last year, including who made each change"

EntitySnapshots

A machine-queryable JSONB snapshot of an entity's key fields captured at every significant status transition (submit, approve, reject, lock, close). This is the structured companion to FormSnapshot — FormSnapshot produces a human-readable PDF stored in S3; EntitySnapshots stores the same moment as queryable data. A caseworker or auditor can ask "what was the enrollment count on this application at the time of approval?" or "what eligibility category did this claim show when it was submitted?" and the system answers via SQL, not by opening a PDF.

Column	Type	Notes
SnapshotId	PK	
TenantCode	FK	cnp_ops schema identifier
EntityType	varchar	e.g. 'Application', 'Claim', 'AdminReview'
EntityId	FK	ID of the snapshotted record

StatusCode	varchar	The status that triggered this snapshot
SnapshotDate	timestamp	
DataJson	JSONB	Key fields of the entity at this moment — queryable via SQL JSONB operators
TriggeredBy	FK → WebUsers	User who caused the status change

Lives in cnp_ops (per-state data). Append-only

FieldChangeLog

Append-only log of individual field edits, capturing intra-status changes that snapshots miss. If a user edits a sponsor's address three times before submitting, EntitySnapshots shows the final state at submission; FieldChangeLog shows all three edits. Together they provide 100% coverage of all data changes as required by the platform spec.

Column	Type	Notes
ChangeId	PK	
TenantCode	FK	
EntityType	varchar	
EntityId	FK	
FieldName	varchar	e.g. 'SponsorAddress', 'ClaimStatus'
OldValue	text	Serialized previous value
NewValue	text	Serialized new value
ChangedBy	FK → WebUsers	
ChangedAt	timestamp	
ChangeSource	varchar	'UI', 'API', or 'System' (batch job, nightly DC load, etc.)

Lives in cnp_ops. Append-only.

manual reconstruction needed.

Notifications

~14 tables · Tier 4 Observational · real-time in-app and outbound alerts

When a claim has been sitting unreviewed for five days, a supervisor needs to know. When a Direct Certification load fails overnight, the state agency DC coordinator needs to know before caseworkers start their morning. When a sponsor's application is approved, the sponsor needs to know. This domain manages every system-generated alert the platform produces — what triggered it, who it was for, how it was delivered, and whether delivery succeeded.

This domain is distinct from Communications & Email. Communications & Email is for human-authored correspondence: a state agency staff member drafting a deadline reminder, a bulk outreach email to all SFSP sponsors. Notifications is for machine-generated alerts that fire automatically based on workflow events, threshold breaches, and scheduled conditions. No human composes a Notification. The platform generates it from a template when a triggering condition is met.

NotificationTemplates defines the alert types: what event triggers the notification, which programs it applies to, what the message says, and whether it should be delivered in-app, by email, or both. NotificationQueue holds pending notifications before they are sent — a decoupled write that allows the platform to continue processing even if the email gateway is temporarily unavailable. NotificationDeliveries logs every send attempt including failures, so a state agency can confirm a notification was actually received and retry if it was not. UserNotificationPreferences lets individual users control which alert types they want and through which channel, within the limits the state agency administrator has configured for their tenant.

Table	What it holds
NotificationTemplates	Template for each alert type: trigger event, program scope, subject, body, delivery channel (in-app / email / both). Lives in cnp_global — templates are shared across all states.
NotificationQueue	Pending notifications: recipient, entity reference, scheduled send time, status. Decoupled from the triggering event so delivery failures don't affect the operational workflow.

NotificationDeliveries	Delivery log: sent timestamp, channel, delivery status, error message if failed. Supports retry logic and delivery confirmation queries.
UserNotificationPrefs	Per-user opt-in/opt-out by alert type and channel. Lives in cnp_ops — per-state user data.

CnpAlerts (in Workflow & Tasks) are internal system conditions — a background job detecting that a review is overdue. Notifications is the outbound layer: CnpAlerts writes to NotificationQueue, which then delivers the message to the user. CnpAlerts is the detection; Notifications is the delivery.

Reference Data & Transparency

~12 tables · Tier 4 Observational · shared lookup values and public reference data

Every dropdown in the platform — program type, eligibility status, finding code, claim status, review type — draws its values from this domain. When a form needs to present a list of valid options, or a compliance reviewer needs to select a finding category, the list comes from here. This is the master repository of all codes and classifications that multiple domains share, kept in cnp_global so every state reads from the same authoritative source.

This domain also holds the reference data USDA publishes for public transparency: poverty line percentages used in eligibility determinations, program participation thresholds, and the canonical list of federal programs the platform supports. These values change infrequently — a poverty line update once a year, a new program added once a decade — and when they do change the update is a database INSERT with an effective date, not a code deployment.

LookupCodes is the single most-read table in the system. Every page load that renders a status badge, a program type label, or a dropdown list reads from it. It lives in cnp_global and is cached aggressively. SystemSettings holds platform-wide configuration values that don't belong to any specific program domain and don't need the feature flag infrastructure of LaunchDarkly — things like default page sizes, session timeout durations, and environment identifiers.

Table	What it holds

LookupCodes	All *Types, *Codes, *Statuses used throughout the system. Category + Code + Description + DisplayOrder. Every dropdown value, every status badge label. Lives in cnp_global. Codes are standardized across all states by design. State-specific display label overrides, if needed, are a separate concern handled at the presentation layer, not additional rows here.
SystemSettings	Platform-wide configuration values: session timeouts, default pagination sizes, environment identifiers. Not tenant-specific, not feature-flagged.
FederalProgramDefinitions	Canonical list of USDA programs the platform supports: NSLP, SBP, SFSP, CACFP, SSFO, FFVP. Almost never changes. A new row would mean USDA created an entirely new federal program. Lives in cnp_global. CEP is a claiming option within NSLP, not a standalone program. It's modeled as an attribute on `ClaimLaws` (CEP_ISP_Pct), not as a row here.
PovertyGuidelineHistory	HHS poverty guidelines by year and household size, effective-dated. Used by eligibility determination when evaluating household income applications. INSERT each year, old rows preserved for audit lookback.

LaunchDarkly

~8 tables · Tier 4 Observational · feature flag and session replay configuration

The platform uses LaunchDarkly as its feature management service. Whether a state has access to the SSFO module, whether non-congregate meal service is enabled for an SFSP tenant, whether Session Replay is capturing sessions for a specific user role — these are controlled through LaunchDarkly flags, not code deployments. This domain is the local database representation of that flag configuration: which flags exist, what their default values are, and which per-tenant overrides are active.

LaunchDarkly itself is an external service. The platform does not re-implement flag evaluation — it calls the LaunchDarkly SDK. This domain exists for two reasons. First, it provides a local cache of flag definitions so that a LaunchDarkly service interruption does not cause the platform to behave unpredictably — flags fall back to their last-known value. Second, it makes flag configuration auditable through the same audit infrastructure as the rest of the platform: a change to a tenant flag override writes to FieldChangeLog, just like a change to any other operational record.

FeatureFlagDefinitions is the registry of all flag keys the platform uses, their type, and their default value. It lives in cnp_global — the flag schema is the same for all states.

TenantFlagOverrides stores the per-state exceptions: Utah has Session Replay enabled at a 10% sample rate; Hawaii has the Training module enabled; Indiana has DirectVer enabled. Each override row records when it was enabled and by whom. SessionReplayConfig extends TenantFlagOverrides for the Session Replay capability specifically, capturing the sample rate, PII masking configuration, and retention policy since these require more structured storage than a simple boolean override.

Table	What it holds
FeatureFlagDefinitions	Registry of all LaunchDarkly flag keys: flag key, type (bool/string/number), default value, description, which module it gates. Lives in cnp_global.
TenantFlagOverrides	Per-state flag values that differ from the default. TenantCode + FlagId + OverrideValue + EnabledAt + EnabledBy. Lives in cnp_ops.
SessionReplayConfig	LaunchDarkly Session Replay settings per tenant: enabled flag, sample rate (0–100%), PII field mask list, session retention period in days.

A Closer Look at the Data

Biggest Table Collapses

In total, roughly 840 of the 1,243 CTS tables were absorbed into roughly 25 greenfield tables. The 413 "Keep" tables are the ones that represent distinct data concepts that couldn't be collapsed further.

LookupCodes 353 → 1 table. The single largest collapse. CTS had one dedicated table per code list (SiteTypes, MealTypes, SponsorTypes, FnsReportTypes, EthnicTypes, etc.). Each one of those becomes a category of rows in a single `LookupCodes` table.

Review Engine 188 → 7 tables. CTS modeled form variation structurally — one table set per review worksheet type. Every new review instrument required new tables. Greenfield stores the same variation as data rows, so all 188 tables fold into `ReviewFormCatalog`, `ReviewQuestions`, `ReviewAnswerOptions`, `ReviewBusinessRules` (cnp_global) plus their instance counterparts (cnp_ops).

Form Engine 99 → 10 tables. Same pattern as reviews, applied to application forms, site detail worksheets, budget worksheets, and annual renewals. One table set per form type in CTS becomes rows in `FormDefinitions`, `FormSections`, `FormFields`, etc.

Fully eliminated 180 tables Temp tables, shuffle tables, version-tracking tables, and CTS deployment artifacts that have no greenfield equivalent at all.

RBAC roughly 20 tables, but also + 5,000+ ACL rows → 6 tables. Not a large table count collapse, but a significant data model collapse The ACL-per-user-per-object pattern is replaced by role-based permissions with only tenant-level overrides where needed.

Tradeoffs/Risks

Referential integrity moves to the API layer. In CTS, `SiteApplications.SiteTypeCode` can FK directly to `SiteTypes.SiteTypeCode`. The database enforces it. With everything collapsed into `LookupCodes`, we can no longer express that constraint in DDL. The same is true for FormFieldValues: all values stored as TEXT, cast at read time. If the API has a bug, invalid data gets in with no database-level backstop.

FormFieldValues and ReviewAnswers will be the largest tables in the system. Every form instance × every field = rows. Across 5 states with heavy claim seasons, this grows fast. The indexing strategy on those two tables needs to be designed deliberately before build, not retrofitted when it slows down.

Reporting complexity increases significantly. Queries that were trivial in CTS ("show all sites that answered Yes to question X") become multi-join Entity-Attribute-Value lookups. State FNS reports often need specific field values aggregated. This isn't a blocker, but whoever writes the reporting layer needs to know they're working against an EAV model, not typed columns. We gain flexibility and maintainability over database-enforced structure, but queries get complex and bad data can slip in if the API has a gap.

The RBAC collapse carries migration risk. 5,000+ ACL rows imply highly granular, per-entity permissions that evolved organically over years. Mapping those to the new role model is where access regressions are most likely to hide as either over-permissive (something opens up that shouldn't) or over-restrictive (a workflow breaks). This mapping needs line-by-line verification against actual production ACL data before cutover, and that verification has to happen onshore since the offshore team can't see production.

The 180 eliminated tables need dependency verification. Temp and version-tracking tables are obvious drops. The risk is stored procedures or scheduled jobs in CTS that reference them. Since the offshore team has zero access to the existing codebase, that dependency check has to be an explicit onshore task before anything is actually dropped.